



Advanced Programming

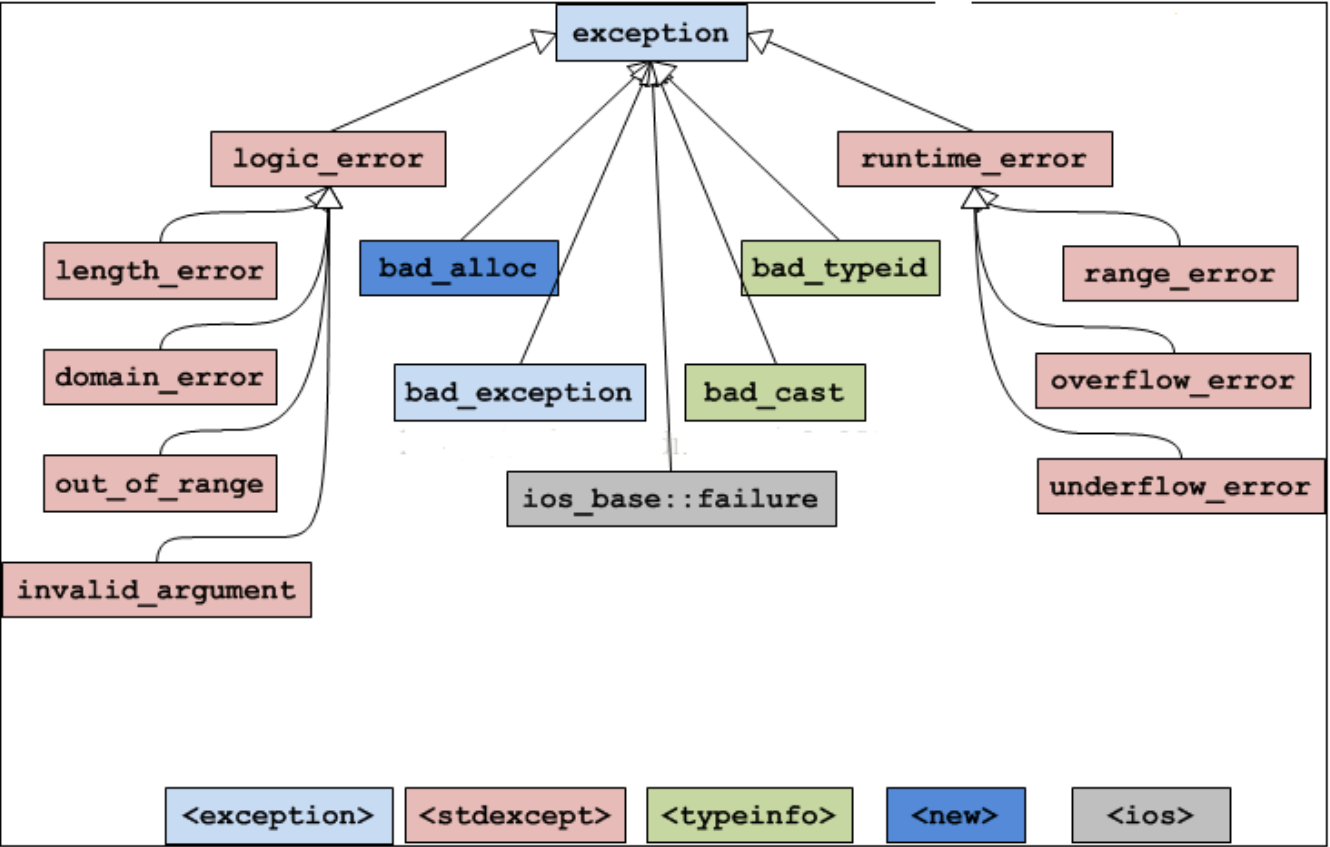
Lab 14, Exceptions, assertion

廖琪梅，王大兴，于仕琪，王薇



C++ Standard Exceptions

C++ provides a list of standard exceptions defined in which we can use in our programs.



Exception	Description
std::exception	An exception and parent class of all the standard C++ exceptions.
std::bad_alloc	This can be thrown by new .
std::bad_cast	This can be thrown by dynamic_cast .
std::bad_exception	This is useful device to handle unexpected exceptions in a C++ program
std::bad_typeid	This can be thrown by typeid.
std::logic_error	An exception that theoretically can be detected by reading the code.
std::domain_error	This is an exception thrown when a mathematically invalid domain is used
std::invalid_argument	This is thrown due to invalid arguments.
std::length_error	This is thrown when a too big std::string is created
std::out_of_range	This can be thrown by the at method from for example a std::vector and std::bitset<>::operator.
std::runtime_error	An exception that theoretically can not be detected by reading the code.
std::overflow_error	This is thrown if a mathematical overflow occurs.
std::range_error	This is occurred when you try to store a value which is out of range.
std::underflow_error	This is thrown if a mathematical underflow occurs.



noexcept, **noexcept(true)**,
and **throw()** are equivalent.

Syntax

noexcept	(1)	
noexcept (<i>expression</i>)	(2)	
throw()	(3)	(deprecated in C++17) (removed in C++20)

```
class exception{  
    public:  
        exception () throw(); //constructor  
        exception (const exception&) throw(); //copy constructor  
        exception& operator= (const exception&) throw(); //assignment operator  
        virtual ~exception() throw(); //destructor  
        virtual const char* what() const throw(); //virtual function  
};
```

Exception specification used in function declaration, with no argument indicates that the function is not allowed to throw any exceptions.

what() is a public method provided by **exception class** which returns a string and it has been overridden by all the child exception classes.



Exercise:

1. Write a function **calculateAverage()** which takes four int arguments, which are marks of four courses, and returns their average as a float number.

The **calculateAverage()** function should take only valid range for marks which is between 0-100. If the marks are out of range throw an **OutOfRangeException** – define this exception as a class.

Invoke the **calculateAverage()** function in main function and get the following inputs and outputs:

```
Please enter marks for 4 courses:70 80 90 67
The average of the four courses is 76.75
Would you want to enter another marks for 4 courses(y/n)?y
Please enter marks for 4 courses:120 56 89 99
The parameter 1 is 120 which out of range(0-100).
Would you want to enter another marks for 4 courses(y/n)?y
Please enter marks for 4 courses:90 -87 67 92
The parameter 2 is -87 which out of range(0-100).
Would you want to enter another marks for 4 courses(y/n)?n
Bye, see you next time.
```



2. Define a generic class “GoldenRectangle” for a golden rectangle in the “golden_rectangle.h”,
When testing based on the following main methods, the test results are shown in the bottom figure.

GoldenRectangle class:

- ✓ two private properties: length and width, and a generic data type.
- ✓ a public method, which returns a Boolean value.
 - ✓ The method calculates by dividing the long side by the short side. If the quotient is 1.618, it returns true; otherwise, it returns false.
- ✓ In this method, If the shorter side is 0 (i.e. the divisor is 0), an assertion is triggered.

```
#include "golden_rectangle.h"
int main() {
    GoldenRectangle<int>rect0(1618, 10);
    std::cout <<"1618x10 is golder Rectangle: "<<std::boolalpha <<rect0.isGolden() <<std::endl;

    GoldenRectangle<int>rect1(34, 21);
    std::cout <<"34x21 is golder Rectangle: "<<std::boolalpha <<rect1.isGolden() <<std::endl;

    GoldenRectangle<double>rect2(1.0, 1.618);
    std::cout <<"1.0x1.618 is golder Rectangle: "<<rect2.isGolden() <<std::endl;

    GoldenRectangle<float>rect3(10.0f, 6.0f);
    std::cout <<"10.0x6.0 is golder Rectangle: "<<rect3.isGolden() <<std::endl;

    GoldenRectangle<int>rect4(10, 0);

    std::cout <<"10x0 is golder Rectangle: "<<rect4.isGolden() <<std::endl;
    return 0;
}
```

```
1618x10 is golder Rectangle: false
34x21 is golder Rectangle: false
1.0x1.618 is golder Rectangle: true
10.0x6.0 is golder Rectangle: false
a.out: golden_rectangle.h:19: bool GoldenRectangle<T>::isGolden() const [with T = int]: Assertion `shorter != 0' failed.
Aborted (core dumped)
```